



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное  
учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ \_\_\_\_\_  
КАФЕДРА \_\_\_\_\_

## КУРСОВАЯ РАБОТА

*НА ТЕМУ:*

*Использование техник быстрого решения  
ограниченных уравнений для сопоставления с  
образцом*

Студент

ИУ9-72Б

(группа)

(подпись, дата)

Терюхов Б.А.

(И.О. Фамилия)

Руководитель курсового  
проекта

(подпись, дата)

Непейвода А.Н.

(И.О. Фамилия)

Консультант

(подпись, дата)

(И.О. Фамилия)

2025 г.

## Оглавление

|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| <i>Введение</i> .....                                                                          | 3  |
| 1.1. Алфавит, слова и шаблоны .....                                                            | 5  |
| 1.2. Подстановка и сопоставление шаблона со словом .....                                       | 8  |
| 1.3. Классы шаблонов .....                                                                     | 11 |
| 1.3.1. Произвольные шаблоны.....                                                               | 11 |
| 1.3.2. Непересекающиеся шаблоны .....                                                          | 12 |
| 1.3.3. Шаблоны с ограниченным числом повторяющихся переменных<br>( $RepV_k$ ).....             | 13 |
| 1.3.4. Шаблоны с ограниченной областью видимости ( $SCD_k$ ).....                              | 14 |
| 1.3.5. Регулярные шаблоны.....                                                                 | 15 |
| 1.3.6. Связь между классами шаблонов .....                                                     | 16 |
| 2. Реализация алгоритмов сопоставления с шаблоном.....                                         | 17 |
| 3. Алгоритмы сопоставления для специализированных классов шаблонов....                         | 20 |
| 3.1. Алгоритм сопоставления для непересекающихся шаблонов .....                                | 20 |
| 3.2. Алгоритм сопоставления для шаблонов с ограниченной областью<br>видимости ( $SCD_k$ )..... | 22 |
| 3.3. Алгоритм сопоставления для регулярных шаблонов .....                                      | 23 |
| 3.4. Алгоритм сопоставления для шаблонов класса $RepV_k$ .....                                 | 24 |
| 4. Экспериментальное сравнение алгоритмов сопоставления.....                                   | 26 |
| 4.1. Методика проведения экспериментов.....                                                    | 26 |
| 4.2. Сравнение наивного и $SCD_k$ матчеров .....                                               | 26 |
| 4.3. Сравнение наивного и $RepV_k$ матчеров .....                                              | 28 |
| 4.4. Сравнение наивного и регулярного матчеров .....                                           | 29 |
| 4.5. Сравнение наивного и непересекающегося матчеров .....                                     | 30 |
| 4.6. Общие выводы по экспериментальному сравнению .....                                        | 31 |
| <i>Результаты курсовой работы</i> .....                                                        | 32 |
| <i>Список использованных источников</i> .....                                                  | 34 |

## Введение

Задача сопоставления строк с шаблонами, содержащими переменные, является одной из фундаментальных задач теории алгоритмов и имеет широкое практическое применение в различных областях компьютерных наук. Подобные задачи возникают при анализе текстов, обработке естественного языка, поиске информации, а также при реализации компиляторов, интерпретаторов и систем автоматического доказательства.

В общем виде задача сопоставления заключается в следующем: по заданному шаблону, состоящему из терминальных символов и переменных, требуется определить, существует ли такая подстановка переменных строками, при которой результат применения подстановки к шаблону совпадает с заданным словом. При этом каждая переменная должна быть заменена одной и той же строкой во всех своих вхождениях.

Наивный подход к решению данной задачи предполагает перебор всех возможных подстановок переменных, что в общем случае приводит к экспоненциальной временной сложности. Несмотря на простоту реализации, такой подход становится неприемлемым уже при умеренных размерах входных данных. В связи с этим представляет интерес разработка и анализ более эффективных алгоритмов сопоставления, использующих структурные ограничения на вид шаблона.

В данной работе рассматриваются несколько классов шаблонов, для которых возможно существенно ускорить процесс сопоставления по сравнению с наивным алгоритмом. В частности, изучаются некрессирующие шаблоны, шаблоны с ограниченной областью видимости переменных, а также регулярные шаблоны. Для каждого класса шаблонов анализируются соответствующие алгоритмы сопоставления, их корректность и временная сложность.

Особое внимание в работе уделяется экспериментальному сравнению различных алгоритмов. Для этого были реализованы генераторы тестовых данных, позволяющие создавать как типичные, так и худшие случаи для каждого алгоритма. Результаты экспериментов представлены в виде графиков и таблиц, что позволяет наглядно сравнить эффективность рассмотренных методов и подтвердить теоретические оценки временной сложности.

Целью данной курсовой работы является разработка, реализация и экспериментальное сравнение алгоритмов сопоставления строк с шаблонами различных классов, а также анализ влияния структурных ограничений шаблонов на эффективность алгоритмов сопоставления.

Для достижения поставленной цели в работе были сформулированы и решены следующие задачи:

- формализовать понятие шаблона и операции сопоставления;
- реализовать наивный алгоритм сопоставления шаблонов;
- разработать оптимизированные алгоритмы для ограниченных классов шаблонов;
- провести экспериментальное сравнение реализованных алгоритмов;
- проанализировать полученные результаты и сформулировать выводы.

## 1.1. Алфавит, слова и шаблоны

В теории формальных языков под *алфавитом* понимается конечное непустое множество символов. В рамках данной работы алфавит обозначается через  $\Sigma$  и используется для построения строк, называемых словами.

### Определение 1.1.

Словом над алфавитом  $\Sigma$  называется конечная последовательность символов из  $\Sigma$ . Множество всех слов над алфавитом  $\Sigma$  обозначается как  $\Sigma^*$ . Длина слова  $w$  обозначается через  $|w|$ .

В данной работе рассматриваются слова, состоящие из терминальных символов, которые сопоставляются с шаблонами, содержащими как терминальные символы, так и переменные.

Для описания шаблонов вводится дополнительное множество переменных, обозначаемое через  $X$ . Переменные представляют собой абстрактные символы, которые могут быть заменены произвольными строками над алфавитом  $\Sigma$ .

### Определение 1.2.

Шаблоном называется конечная последовательность элементов, каждый из которых является либо терминальным символом из алфавита  $\Sigma$ , либо переменной из множества  $X$ .

Таким образом, шаблон можно рассматривать как строку над расширенным алфавитом  $\Sigma \cup X$ .

В отличие от обычных строк, шаблоны допускают множественные вхождения одних и тех же переменных. При этом все вхождения одной переменной при сопоставлении должны быть заменены одной и той же строкой.

### Пример 1.1.

Пусть  $\Sigma = \{a, b\}$ , а  $X = \{x_1, x_2\}$ . Тогда выражение

$$x_1 a x_2 x_1$$

является шаблоном. При подстановке  $x_1 \rightarrow ab$  и  $x_2 \rightarrow b$  результатом применения шаблона будет слово

$$ababab$$

Для удобства дальнейшего изложения введём различие между терминальными и переменными элементами шаблона.

### Определение 1.3.

Элемент шаблона называется *терминальным*, если он принадлежит алфавиту  $\Sigma$ , и *переменным*, если он принадлежит множеству  $X$ .

Шаблон может содержать как одиночные терминальные символы, так и последовательности переменных. В ряде алгоритмов имеет смысл рассматривать шаблон как последовательность блоков одинаковых переменных или терминальных символов.

### Определение 1.4.

Блоком переменной называется максимальная непрерывная подпоследовательность шаблона, состоящая из одинаковых переменных.

Наличие повторяющихся блоков переменных существенно влияет на сложность задачи сопоставления. В частности, чем больше количество переменных и чем чаще они повторяются в шаблоне, тем больше возможных подстановок необходимо рассмотреть при наивном подходе.

Следует отметить, что в общем случае задача сопоставления шаблона со словом является вычислительно сложной и требует перебора большого количества вариантов подстановок. Однако введение структурных ограничений на вид шаблона, таких как запрет пересечения областей видимости переменных или ограничение количества активных переменных, позволяет существенно упростить задачу и разработать более эффективные алгоритмы.

В последующих разделах данной работы будут подробно рассмотрены различные классы шаблонов и соответствующие им алгоритмы сопоставления.

## 1.2. Подстановка и сопоставление шаблона со словом

Для формального описания процесса сопоставления шаблона со словом необходимо ввести понятие подстановки переменных и операции применения подстановки к шаблону.

### Определение 1.5.

Подстановкой называется отображение

$$\theta: X \rightarrow \Sigma^*$$

которое каждой переменной из множества  $X$  ставит в соответствие некоторое слово над алфавитом  $\Sigma$ .

Подстановка может быть как полной, так и частичной. В рамках данной работы рассматривается задача **полного сопоставления** шаблона со словом. Это означает, что подстановка считается успешной лишь в том случае, если результат применения подстановки к шаблону полностью совпадает с заданным словом, а длина полученной строки равна длине слова. Иными словами, для успешного сопоставления требуется, чтобы

$$\theta(p) = w,$$

и не допускается наличие непокрытых символов ни в шаблоне, ни в слове. Частичное сопоставление и поиск вхождений шаблона внутри слова в рамках данной работы не рассматриваются.

### Определение 1.6.

Применением подстановки  $\theta$  к шаблону  $p$  называется операция, в результате которой каждая переменная  $x \in X$ , входящая в шаблон, заменяется на слово  $\theta(x)$ , если оно определено, либо остаётся неизменной в противном случае.

Результатом применения подстановки к шаблону является строка над алфавитом  $\Sigma$ , если подстановка определена для всех переменных шаблона.

### Пример

1.2.

Пусть задан шаблон

$$p = x_1 b x_2 x_1$$

и подстановка

$$\theta(x_1) = a, \theta(x_2) = bb.$$

Тогда результат применения подстановки равен слову

$$abbba$$

Основной задачей сопоставления шаблона со словом является определение существования такой подстановки, при которой шаблон после подстановки совпадает с заданным словом.

### Определение 1.7.

Подстановка  $\theta$  называется согласованной, если для любой переменной  $x \in X$  и для любых двух её вхождений в шаблоне значения подстановки совпадают.

Это ограничение существенно усложняет задачу сопоставления по сравнению с классическим поиском подстроки или регулярными выражениями, так как приводит к появлению зависимостей между удалёнными участками шаблона.

Следует отметить, что длины подстановок переменных заранее неизвестны и могут существенно различаться. В общем случае длина подстановки переменной может принимать любое неотрицательное целое

значение, что приводит к необходимости перебора возможных вариантов при наивном подходе.

В процессе сопоставления часто используется понятие частичного совпадения, при котором анализируется лишь префикс шаблона и соответствующий ему префикс слова.

### 1.3. Классы шаблонов

В общем случае задача сопоставления шаблона со словом допускает произвольное расположение переменных и терминальных символов в шаблоне. Однако именно структура шаблона в наибольшей степени влияет на сложность задачи сопоставления. В данной работе рассматриваются несколько классов шаблонов, для которых возможно применение более эффективных алгоритмов по сравнению с наивным перебором подстановок.

#### 1.3.1. Произвольные шаблоны

##### Определение 1.10.

Произвольным шаблоном называется шаблон, в котором переменные могут встречаться в любом порядке и произвольное количество раз, без дополнительных структурных ограничений.

В произвольных шаблонах области видимости различных переменных могут пересекаться, а повторные вхождения переменных могут чередоваться друг с другом.

##### Пример 1.3.

Шаблон

$$x_1 x_2 x_1 x_2$$

является произвольным, так как области видимости переменных  $x_1$  и  $x_2$  пересекаются.

Для данного класса шаблонов не существует известных детерминированных алгоритмов сопоставления с полиномиальной временной сложностью в общем случае. Наивный алгоритм сопоставления для произвольных шаблонов основан на переборе возможных подстановок

переменных и имеет экспоненциальную временную сложность в худшем случае. Именно произвольные шаблоны рассматриваются в качестве наиболее общего случая и используются в данной работе для демонстрации ограничений и недостатков наивного подхода.

### 1.3.2. Непересекающиеся шаблоны

Одним из важных подклассов произвольных шаблонов являются непересекающиеся шаблоны.

#### Определение 1.11.

Шаблон называется непересекающимся, если области видимости любых двух различных переменных не пересекаются.

Областью видимости переменной называется отрезок шаблона от первого до последнего вхождения данной переменной. Формально, для любых двух различных переменных  $x$  и  $y$  с областями видимости  $[l_x, r_x]$  и  $[l_y, r_y]$  выполняется одно из условий:

$$r_x < l_y \text{ или } r_y < l_x$$

#### Пример 1.4.

Шаблон

$$x_1 \ x_1 \ a \ x_2 \ x_2 \ b \ x_3$$

является непересекающимся, так как области видимости переменных не пересекаются.

Непересекающиеся шаблоны обладают важным свойством: каждую переменную можно рассматривать независимо от других, так как её вхождения образуют один непрерывный блок. Это позволяет определить

длину подстановки переменной детерминированно, исходя из контекста шаблона, без перебора возможных значений.

В результате для непересекающихся шаблонов возможно построение алгоритмов сопоставления с линейной или квазилинейной временной сложностью, что делает их существенно более эффективными по сравнению с наивным алгоритмом.

### 1.3.3. Шаблоны с ограниченным числом повторяющихся переменных ( $RepV_k$ )

В ряде случаев сложность сопоставления шаблона со словом определяется не столько структурой областей видимости переменных, сколько количеством переменных, которые встречаются в шаблоне более одного раза. Для формализации такого ограничения вводится класс шаблонов  $RepV_k$ .

#### Определение 1.14.

Шаблон принадлежит классу  $RepV_k$ , если количество переменных, имеющих более одного вхождения в шаблоне, не превышает  $k$ . Иными словами, в шаблоне может присутствовать произвольное число переменных, однако не более  $k$  из них могут повторяться. Остальные переменные встречаются в шаблоне ровно один раз.

#### Пример 1.7.

Шаблон

$$x_1 \ a \ x_2 \ b \ x_1 \ c \ x_3$$

принадлежит классу  $RepV_1$ , так как только переменная  $x_1$  встречается более одного раза.

Ограничение  $RepV_k$  существенно снижает сложность задачи сопоставления, поскольку именно повторяющиеся переменные создают зависимости между различными частями шаблона. При фиксированном

значении  $k$  возможно построение алгоритмов сопоставления, которые перебирают подстановки только для повторяющихся переменных, в то время как переменные с единственным вхождением могут обрабатываться детерминированно.

Таким образом, класс  $RepV_k$  является промежуточным между регулярными и произвольными шаблонами. Он допускает повторение переменных, но в строго ограниченном количестве, что позволяет существенно сократить пространство поиска по сравнению с наивным алгоритмом.

#### 1.3.4. Шаблоны с ограниченной областью видимости ( $SCD_k$ )

В более общем случае допускается частичное пересечение областей видимости переменных. Для описания таких шаблонов используется понятие *ограниченной области видимости*.

#### Определение 1.15.

Говорят, что шаблон обладает свойством  $Scd_k$  (Scope Coincidence Degree  $k$ ), если в любой позиции шаблона одновременно активны не более  $k$  различных переменных. Переменная считается активной в позиции шаблона, если эта позиция находится внутри её области видимости.

#### Пример 1.8.

Шаблон

$$x_1 \ x_2 \ x_1 \ x_2$$

обладает свойством  $Scd_2$ , так как в каждой позиции активны не более двух переменных.

Шаблоны с ограниченной областью видимости занимают промежуточное положение между произвольными и непересекающимися шаблонами. С одной стороны, они допускают пересечение областей видимости переменных, а с другой — накладывают ограничение на максимальное число одновременно активных переменных.

Для шаблонов класса  $Scd_k$  возможно построение алгоритмов сопоставления, которые используют данное ограничение для отсечения части пространства поиска. При фиксированном значении  $k$  такие алгоритмы имеют существенно лучшую практическую эффективность по сравнению с наивным алгоритмом.

### 1.3.5. Регулярные шаблоны

Отдельным важным классом являются регулярные шаблоны.

#### Определение 1.16.

Регулярным шаблоном называется шаблон, в котором каждая переменная встречается не более одного раза.

#### Пример 1.9.

Шаблон

$$x_1 \ a \ x_2 \ b \ x_3$$

является регулярным.

Регулярные шаблоны не содержат повторных вхождений переменных, а значит, не накладывают ограничений согласованности подстановок. В этом случае задача сопоставления шаблона со словом сводится к разбиению слова на фрагменты, разделённые терминальными символами шаблона.

Для регулярных шаблонов возможна реализация особенно эффективных алгоритмов сопоставления, близких по сложности к алгоритмам сопоставления регулярных выражений, однако с учётом того, что переменные могут сопоставляться произвольным строкам.

#### 1.3.6. Связь между классами шаблонов

Рассмотренные классы шаблонов образуют иерархию по степени общности:

$$\text{Регулярные} \subset \text{Rep}V_k \subset \text{SCD}_k \subset \text{Произвольные}.$$

Следует отметить, что ограничение непересекающихся шаблонов носит ортогональный характер по отношению к ограничениям  $\text{Rep}V_k$  и  $\text{SCD}_k$ . В то время как классы  $\text{Rep}V_k$  и  $\text{SCD}_k$  накладывают количественные ограничения на повторяемость и активность переменных, непересекающиеся шаблоны вводят структурное ограничение на взаимное расположение областей видимости переменных. В результате класс непересекающихся шаблонов не образует линейной иерархии с другими рассмотренными классами и может пересекаться с каждым из них. Чем более строгие ограничения накладываются на структуру шаблона, тем более эффективные алгоритмы сопоставления могут быть построены. В последующих главах данной работы для каждого из перечисленных классов шаблонов рассматриваются соответствующие алгоритмы сопоставления и проводится их экспериментальное сравнение.

## 2. Реализация алгоритмов сопоставления с шаблоном

Наивный алгоритм сопоставления шаблона с переменными представляет собой прямолинейную реализацию определения существования подстановки, при которой шаблон полностью совпадает с заданным словом. Под переменной в шаблоне понимается символ, которому может быть сопоставлена произвольная подстрока слова, включая пустую строку. Терминальные символы шаблона должны совпадать с соответствующими символами слова строго посимвольно. Алгоритм ориентирован на **полное сопоставление**, то есть шаблон считается сопоставленным со словом только в том случае, если:

- обработаны все элементы шаблона;
- при этом достигнут конец слова.

Наивный подход не использует дополнительных эвристик или структурных ограничений и опирается исключительно на перебор возможных значений переменных с возвратом (backtracking).

Основные особенности наивного алгоритма:

- переменные могут принимать любые подстроки слова;
- повторные вхождения переменной обязаны соответствовать одному и тому же значению;
- перебор возможных значений осуществляется в порядке увеличения длины подстроки;
- при несоответствии происходит возврат к предыдущему состоянию подстановки.

Таким образом, наивный алгоритм служит **базовой точкой отсчёта** для сравнения эффективности более оптимизированных методов сопоставления.

Реализация наивного алгоритма сопоставления шаблона с переменными выполнена в виде рекурсивной функции с явным управлением позициями в шаблоне и слове.

Основной метод `match` инициализирует процесс сопоставления и вызывает рекурсивную функцию `matchFrom`, передавая начальные позиции шаблона и слова, равные нулю.

```
override fun match(  
    pattern: Pattern,  
    word: Word,  
    substitution: MutableMap<String, String>  
) : Substitution? {  
    return matchFrom(pattern, word, substitution, 0, 0)  
}
```

Рекурсивная функция `matchFrom` принимает текущие позиции в шаблоне (`pPos`) и слове (`wPos`), а также текущее состояние подстановки переменных.

#### Базовый случай рекурсии

Если достигнут конец шаблона, алгоритм проверяет, достигнут ли одновременно конец слова. Это гарантирует выполнение условия полного сопоставления.

```
if (pPos == pattern.size) {  
    return if (wPos == word.length) substitution else  
    null  
}
```

Если слово не полностью использовано, сопоставление считается неуспешным.

### Обработка терминального символа

Если текущий элемент шаблона является терминальным символом, выполняется проверка на совпадение соответствующего символа слова.

```
if (wPos < word.length && word[wPos] == el.symbol) {  
    matchFrom(pattern, word, substitution, pPos + 1, wPos  
+ 1)  
}
```

При несовпадении алгоритм немедленно завершает текущую ветвь поиска.

### Обработка переменной

Если текущий элемент шаблона является переменной, возможны два случая.

#### Повторное вхождение переменной

Если переменная уже имеет значение в текущей подстановке, алгоритм проверяет, начинается ли слово в текущей позиции с этого значения.

```
if (current != null) {  
    if (word.startsWith(current, wPos)) {  
        matchFrom(pattern, word, substitution, pPos + 1,  
wPos + current.length)  
    }  
}
```

Таким образом обеспечивается корректность повторных вхождений переменных.

#### Первое вхождение переменной

Если переменная встречается впервые, алгоритм перебирает **все возможные подстроки**, начиная с текущей позиции слова.

```
for (len in 0..(word.length - wPos)) {  
    val candidate = word.substring(wPos, wPos + len)  
    substitution[name] = candidate  
    val res = matchFrom(...)  
    if (res != null) return res  
    substitution.remove(name)  
}
```

Каждое возможное значение подставляется рекурсивно, а при неудаче происходит возврат состояния подстановки.

### Характеристика алгоритма

Данная реализация обладает следующими свойствами:

- алгоритм корректен для всех шаблонов без структурных ограничений;
- поддерживает пустые значения переменных;
- использует полный перебор возможных подстановок;
- имеет экспоненциальную временную сложность в худшем случае.

Именно из-за экспоненциального характера перебора данный алгоритм становится неэффективным при увеличении длины слова и количества переменных, что делает его удобным эталоном для сравнения с оптимизированными матчерами, реализованными в последующих разделах работы.

## 3. Алгоритмы сопоставления для специализированных классов шаблонов

### 3.1. Алгоритм сопоставления для непересекающихся шаблонов

Алгоритм сопоставления для непересекающихся шаблонов использует структурное свойство шаблона: области видимости различных переменных не пересекаются. Это позволяет отказаться от обработки шаблона как последовательности отдельных элементов и перейти к работе с укрупнёнными структурными блоками.

### Общая идея алгоритма

Шаблон предварительно разбивается на последовательность сегментов двух типов:

- сегменты терминальных символов;
- сегменты переменных, содержащие один или несколько подряд идущих блоков повторяющихся переменных.

Каждый сегмент обрабатывается последовательно слева направо. Поскольку области видимости переменных не пересекаются, все вхождения одной переменной находятся внутри одного сегмента и могут обрабатываться как единое целое.

### Основные шаги алгоритма

#### 1. Разбиение шаблона на сегменты.

Последовательно просматривается шаблон и формируются сегменты вида

(переменные) (терминалы) (переменные) ...

#### 2. Последовательное сопоставление сегментов.

Алгоритм рекурсивно сопоставляет каждый сегмент со словом, поддерживая текущую позицию в слове.

#### 3. Обработка сегмента терминалов.

Выполняется прямое сравнение соответствующей подстроки слова с

терминальным сегментом. При несовпадении текущая ветвь поиска отсекается.

#### **4. Обработка сегмента переменных.**

Для каждого блока переменной подбирается длина её значения таким образом, чтобы все повторные вхождения переменной совпадали. После успешного сопоставления сегмента позиция в слове сдвигается на длину обработанного фрагмента.

#### **5. Завершение сопоставления.**

Сопоставление считается успешным, если обработаны все сегменты и достигнут конец слова.

За счёт отсутствия пересечений областей видимости backtracking применяется только на уровне сегментов, что существенно снижает количество перебираемых вариантов.

### **3.2. Алгоритм сопоставления для шаблонов с ограниченной областью видимости ( $SCD_k$ )**

Алгоритм сопоставления для шаблонов класса  $SCD_k$  основан на ограничении максимального числа переменных, одновременно активных в любой позиции шаблона.

#### **Общая идея алгоритма**

Алгоритм выполняет сопоставление шаблона со словом слева направо, поддерживая информацию о текущих активных переменных. Ограничение  $SCD_k$  позволяет отсекаать ветви поиска, в которых число активных переменных превышает допустимое значение  $k$ .

#### **Основные шаги алгоритма**

##### **1. Инициализация.**

Устанавливаются начальные позиции в шаблоне и слове, подстановка переменных пуста.

## **2. Последовательная обработка элементов шаблона.**

Для каждого элемента шаблона выполняется одна из следующих операций:

- при термине проверяется совпадение символа слова;
- при переменной проверяется наличие уже назначенного значения.

## **3. Обработка первого вхождения переменной.**

Если переменная встречается впервые, перебираются возможные значения, начиная с текущей позиции слова. Перед началом перебора проверяется, не превышает ли число активных переменных параметр  $k$ .

## **4. Обработка повторного вхождения переменной.**

Проверяется, что слово в текущей позиции начинается с ранее назначенного значения переменной.

## **5. Отсечение недопустимых состояний.**

Если в какой-либо момент число активных переменных превышает  $k$ , текущая ветвь поиска немедленно отбрасывается.

## **6. Завершение сопоставления.**

Сопоставление успешно, если достигнут конец шаблона и конец слова.

Благодаря локальному контролю числа активных переменных алгоритм избегает экспоненциального роста пространства поиска, характерного для наивного подхода.

### **3.3. Алгоритм сопоставления для регулярных шаблонов**

Алгоритм сопоставления для регулярных шаблонов существенно упрощён по сравнению с общим случаем, поскольку каждая переменная встречается в шаблоне ровно один раз.

#### **Общая идея алгоритма**

Отсутствие повторных вхождений переменных устраняет необходимость проверки согласованности подстановок. Каждая переменная может быть

сопоставлена с произвольной подстрокой слова единожды, без дальнейших ограничений.

#### Основные шаги алгоритма

**1. Последовательный проход по шаблону.**

Шаблон и слово обрабатываются слева направо.

**2. Обработка терминалов.**

Выполняется прямое сравнение соответствующих символов слова.

**3. Обработка переменных.**

Переменной присваивается подстрока слова, длина которой определяется оставшейся длиной слова и количеством элементов шаблона.

**4. Завершение сопоставления.**

Сопоставление считается успешным при одновременном достижении конца шаблона и слова.

Поскольку backtracking по повторным переменным отсутствует, алгоритм имеет существенно меньшую вычислительную сложность и близок по эффективности к линейному.

### 3.4. Алгоритм сопоставления для шаблонов класса $\text{Rep}V_k$

Алгоритм сопоставления для шаблонов класса  $\text{Rep}V_k$  использует ограничение на количество переменных, имеющих повторные вхождения.

#### Общая идея алгоритма

Основная вычислительная сложность сопоставления связана с повторяющимися переменными, поскольку они создают зависимости между различными частями шаблона. В классе  $\text{Rep}V_k$  число таких переменных ограничено, что позволяет явно выделить их и обработать отдельно.

## Основные шаги алгоритма

### 1. Выделение повторяющихся переменных.

Определяются переменные, встречающиеся в шаблоне более одного раза. Если их число превышает  $k$ , сопоставление прекращается.

### 2. Формирование уравнения длины.

Для каждой переменной учитывается количество её вхождений в шаблоне, что позволяет задать ограничение на длины значений переменных.

### 3. Перебор допустимых длин значений.

Рекурсивно перебираются возможные длины значений повторяющихся переменных, удовлетворяющие суммарному ограничению длины слова.

### 4. Проверка конкретного сопоставления.

Для каждого набора длин выполняется линейный проход по шаблону и слову с проверкой согласованности значений переменных.

### 5. Завершение сопоставления.

Алгоритм возвращает первую найденную корректную подстановку либо сообщает об отсутствии сопоставления.

За счёт ограничения на число повторяющихся переменных перебор существенно сокращается по сравнению с наивным алгоритмом.

## 4. Экспериментальное сравнение алгоритмов сопоставления

### 4.1. Методика проведения экспериментов

Для оценки эффективности реализованных алгоритмов сопоставления были проведены вычислительные эксперименты, направленные на сравнение времени работы наивного алгоритма и специализированных матчеров для различных классов шаблонов.

В ходе экспериментов измерялось время выполнения сопоставления шаблона со словом в зависимости от длины входных данных. Для каждого алгоритма использовались специально сгенерированные пары «шаблон–слово», удовлетворяющие ограничениям соответствующего класса шаблонов. Длина слов варьировалась от нескольких тысяч до сотен тысяч символов.

Измерение времени выполнения проводилось с использованием аннотации @MeasureTime и класса TimeMeasurer с методом measure что позволило получать значения времени работы алгоритмов в миллисекундах. Для наглядности результаты были визуализированы в виде графиков зависимости времени выполнения от длины входных данных.

### 4.2. Сравнение наивного и SCD<sub>k</sub> матчеров

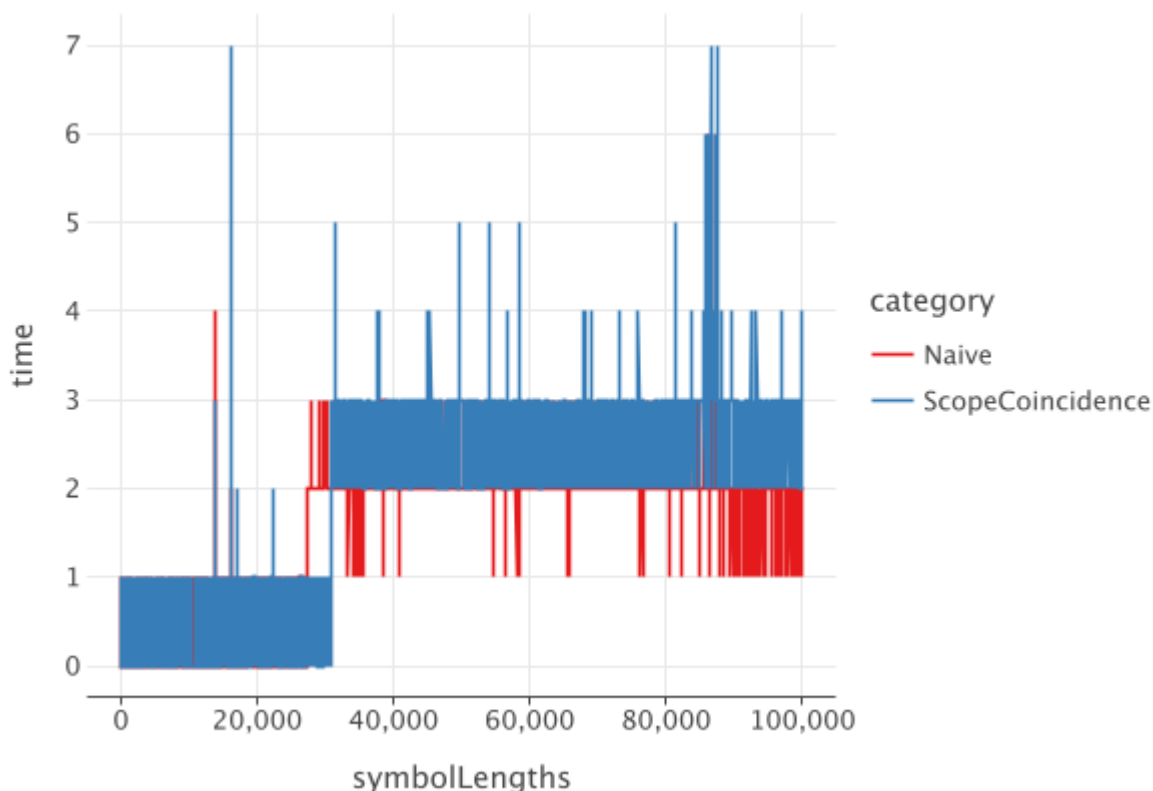


Рисунок 1.1 - Сравнение производительности наивного и SCD матчеров

На первом графике представлено сравнение времени работы наивного алгоритма и алгоритма сопоставления для шаблонов класса  $SCD_k$ .

Для малых размеров входных данных время работы обоих алгоритмов сопоставимо. Это объясняется тем, что накладные расходы на дополнительные проверки в  $SCD_k$  матчер (учёт активных переменных и контроль области видимости) на коротких строках превосходят выигрыш от уменьшения пространства поиска.

Однако с ростом длины входных данных наблюдается принципиальное различие в поведении алгоритмов. Время работы наивного алгоритма демонстрирует значительную вариативность и резкие скачки, обусловленные экспоненциальным ростом числа перебираемых подстановок. В то же время  $SCD_k$  матчер показывает более стабильное время выполнения, что подтверждает эффективность отсечения ветвей поиска при превышении допустимого числа активных переменных.

Таким образом, экспериментально подтверждается, что ограничение области видимости переменных позволяет существенно сократить пространство поиска и повысить устойчивость алгоритма на больших входных данных.

#### 4.3. Сравнение наивного и RepV<sub>k</sub> матчеров

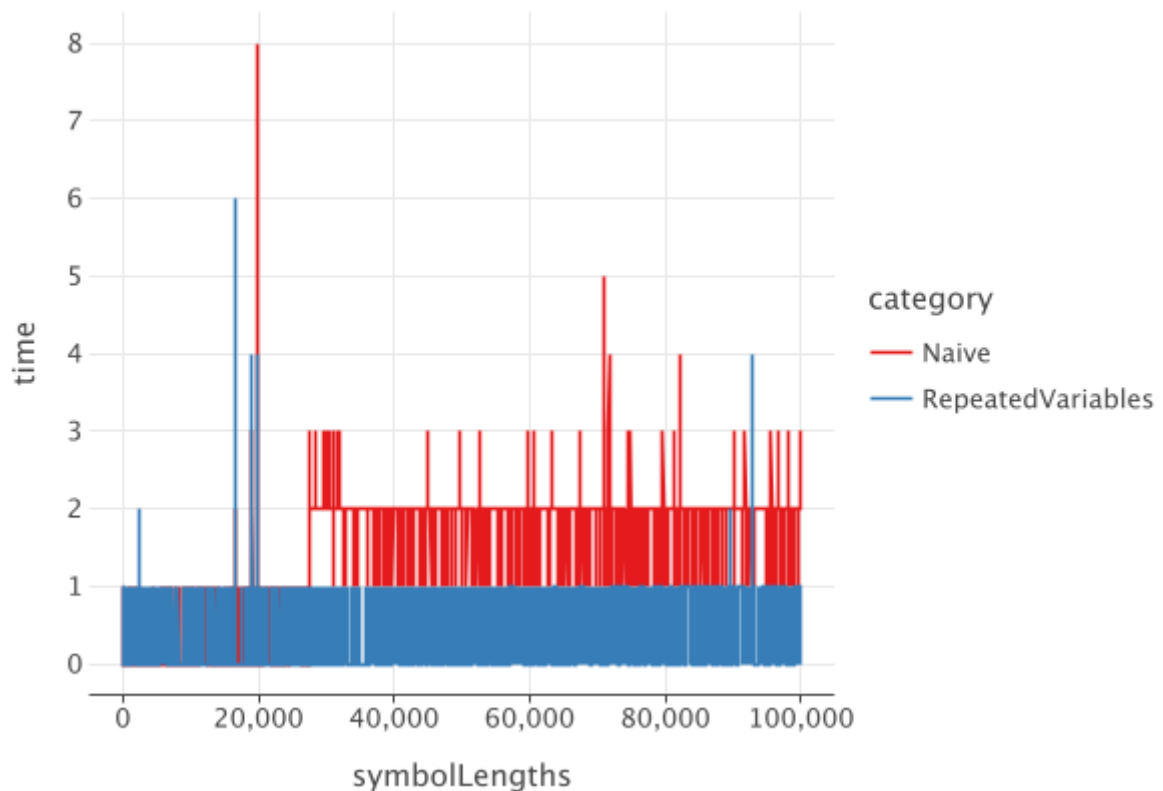


Рисунок 1.2 - Сравнение производительности наивного и RepV<sub>k</sub> матчеров

График иллюстрирует сравнение наивного алгоритма с алгоритмом сопоставления для шаблонов класса RepV<sub>k</sub>.

Полученные результаты показывают, что RepV<sub>k</sub> матчер демонстрирует значительно более равномерное время выполнения по сравнению с наивным алгоритмом. Это связано с тем, что в алгоритме RepV<sub>k</sub> перебор осуществляется не по всем возможным подстрокам, а по допустимым длинам значений повторяющихся переменных, число которых ограничено параметром  $k$ .

Наивный алгоритм, напротив, при увеличении длины входных данных всё чаще сталкивается с необходимостью глубокого backtracking, что приводит к резким увеличениям времени работы.

Эксперимент подтверждает, что ограничение на число повторяющихся переменных является эффективным способом снижения вычислительной сложности сопоставления.

#### 4.4. Сравнение наивного и регулярного матчеров

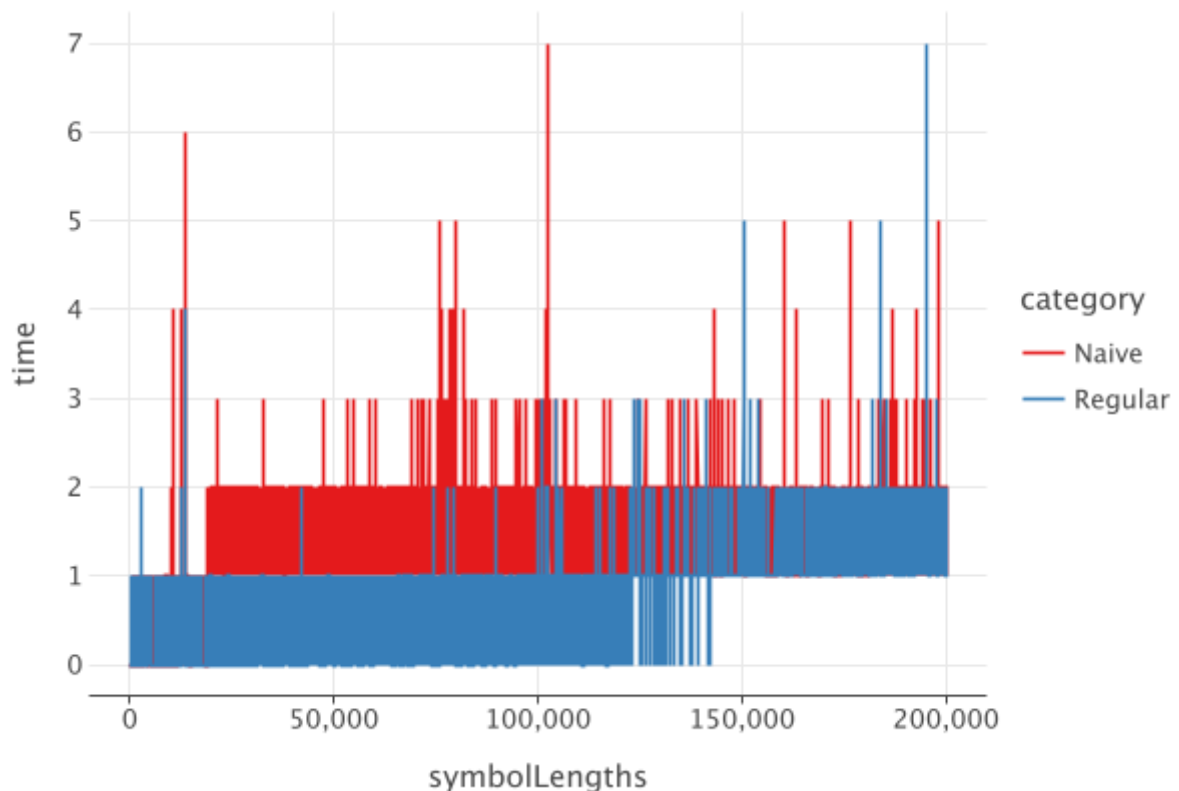


Рисунок 1.3 - Сравнение производительности наивного и регулярного матчеров

График отражает сравнение наивного алгоритма и алгоритма сопоставления для регулярных шаблонов.

Регулярный матчер демонстрирует наилучшую масштабируемость среди всех рассмотренных алгоритмов. Время его работы растёт практически линейно с увеличением длины входных данных. Это объясняется отсутствием повторных вхождений переменных и, как следствие, отсутствием необходимости проверять согласованность подстановок.

Наивный алгоритм, хотя и показывает приемлемые результаты на коротких строках, при увеличении длины входных данных становится значительно менее предсказуемым, что подтверждает теоретический анализ его экспоненциальной сложности.

#### 4.5. Сравнение наивного и непересекающегося матчеров

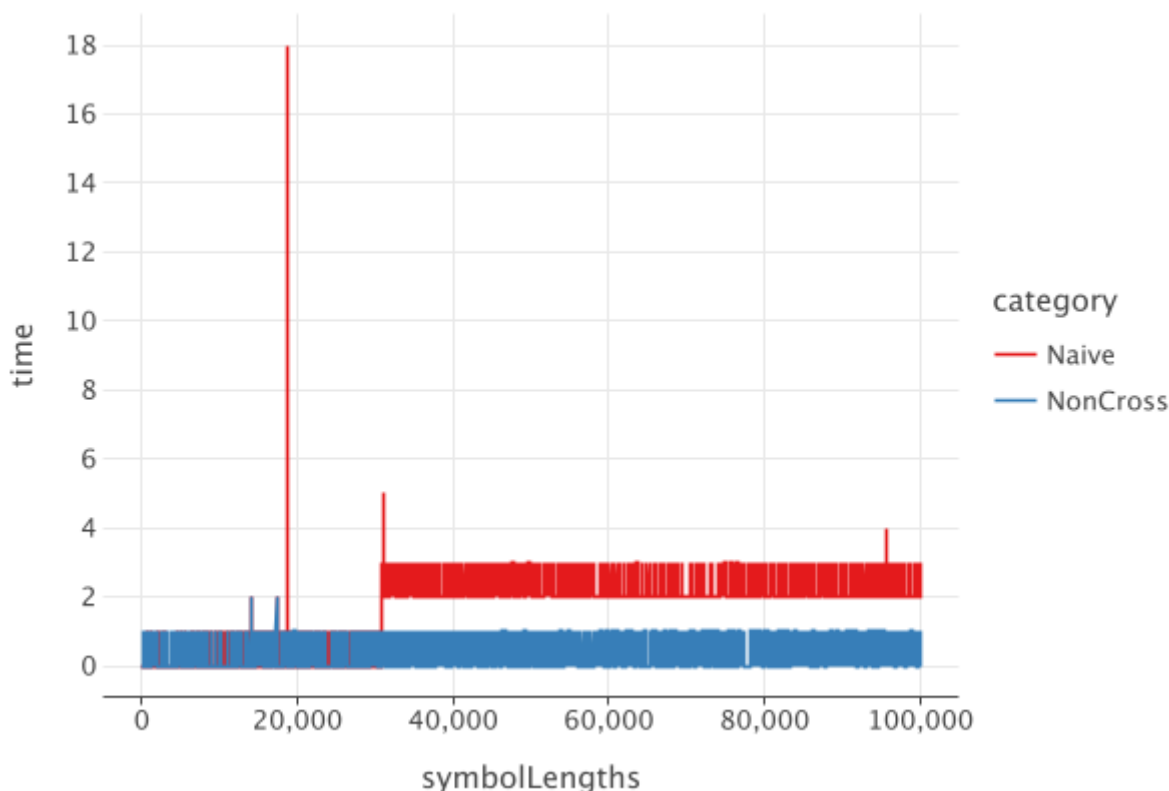


Рисунок 1.4 - Сравнение производительности наивного и непересекающегося матчеров

На графике представлено сравнение наивного алгоритма и алгоритма сопоставления для непересекающихся шаблонов.

Результаты показывают, что непересекающийся матчер существенно превосходит наивный алгоритм при увеличении длины входных данных. Это связано с тем, что непересекающаяся структура шаблона позволяет обрабатывать повторные вхождения переменных в виде цельных блоков и исключить необходимость глобального backtracking.

Особенно заметна разница в худших случаях: наивный алгоритм демонстрирует резкие пики времени выполнения, тогда как

непересекающийся матчер сохраняет стабильное и ограниченное время работы.

#### **4.6. Общие выводы по экспериментальному сравнению**

Проведённые эксперименты подтверждают, что структурные и количественные ограничения на шаблоны позволяют существенно повысить эффективность алгоритмов сопоставления по сравнению с наивным подходом.

Основные выводы:

- наивный алгоритм пригоден лишь для малых входных данных и служит базовой точкой сравнения;
- ограничение области видимости ( $SCD_k$ ) эффективно уменьшает пространство поиска;
- ограничение числа повторяющихся переменных ( $RepV_k$ ) даёт устойчивый выигрыш по времени;
- регулярные шаблоны допускают наиболее эффективные алгоритмы сопоставления;
- непересекающиеся шаблоны позволяют избежать экспоненциального роста времени выполнения.

Таким образом, экспериментальные результаты полностью согласуются с теоретическим анализом и подтверждают целесообразность введения специализированных классов шаблонов и соответствующих алгоритмов сопоставления.

## Результаты курсовой работы

В рамках данной курсовой работы была рассмотрена задача сопоставления строк с шаблонами, содержащими переменные, и исследованы алгоритмы её решения при различных структурных и количественных ограничениях на шаблоны.

В ходе работы были получены следующие основные результаты:

1. **Проанализирована задача сопоставления шаблонов с переменными** и показано, что наивный алгоритм, основанный на полном переборе возможных подстановок, обладает экспоненциальной временной сложностью и становится непрактичным при увеличении длины входных данных.
2. **Рассмотрены основные классы шаблонов**, включая:
  - регулярные шаблоны;
  - шаблоны с ограниченным числом повторяющихся переменных ( $\text{Rep}V_k$ );
  - шаблоны с ограниченной областью видимости переменных ( $\text{SCD}_k$ );
  - непересекающиеся шаблоны.
3. **Разработаны и реализованы алгоритмы сопоставления** для каждого из перечисленных классов шаблонов. Реализации выполнены на языке Kotlin и опираются на структурные свойства шаблонов, что позволяет существенно сократить пространство поиска по сравнению с наивным методом.
4. **Проведено экспериментальное сравнение** времени работы наивного алгоритма и специализированных матчеров. Эксперименты показали, что:

- наивный алгоритм демонстрирует нестабильное время выполнения и резкие пики при увеличении длины входных данных;
- специализированные алгоритмы обладают более предсказуемым и устойчивым временем работы;
- наибольший выигрыш достигается для регулярных и непересекающихся шаблонов;
- ограничения  $RepV_k$  и  $SCD_k$  позволяют существенно снизить вычислительную сложность при сохранении полноты сопоставления.

**5. Экспериментальные результаты полностью согласуются с теоретическим анализом,** подтверждая, что введение структурных и количественных ограничений на шаблоны является эффективным способом оптимизации задачи сопоставления.

Таким образом, в работе показано, что использование специализированных алгоритмов сопоставления для различных классов шаблонов позволяет добиться значительного улучшения производительности по сравнению с наивным подходом, что делает данные методы применимыми для обработки больших входных данных.

## Список использованных источников

1. Florin Manea<sup>1</sup> and Markus L. Schmid<sup>2</sup>. Matching Patterns with Variables [учебное пособие] // Сопоставление строк с шаблонами, содержащими переменные
2. Документация языка Kotlin [Электронный ресурс] - URL: <https://kotlinlang.org/docs/home.html> (дата обращения 15.09.2025)